

Hoare Logic II

Invariants and weakest preconditions

Cristina David

Lecture 9

COMSM0067 — Advanced Topics in Programming Languages

University of Bristol

- A third semantics: the Hoare triple $\{P\} c \{Q\}$, and **partial** correctness.
- Six rules; Conseq the seam where logic enters.
- The assignment axiom runs **backwards**, so the swap proof **wrote itself**, right to left.

Last time

- A third semantics: the Hoare triple $\{P\} c \{Q\}$, and **partial** correctness.
- Six rules; Conseq the seam where logic enters.
- The assignment axiom runs **backwards**, so the swap proof **wrote itself**, right to left.

And we ended on **one** unanswered question.

1. **Where does the invariant come from?**

Last time

- A third semantics: the Hoare triple $\{P\} c \{Q\}$, and **partial** correctness.
- Six rules; Conseq the seam where logic enters.
- The assignment axiom runs **backwards**, so the swap proof **wrote itself**, right to left.

And we ended on **one** unanswered question.

1. **Where does the invariant come from?**

Today: **Invariants wp** and **VC generation** answer this.

Loop invariants

The rule that doesn't tell you how

$$\frac{\vdash \{I \wedge b\} c \{I\}}{\vdash \{I\} \text{ while } b \text{ do } c \{I \wedge \neg b\}} \text{ While}$$

Everything hinges on I . But the rule is not **syntax-directed** in I : it does not compute the invariant, it only **checks** one you supply.

The rule that doesn't tell you how

$$\frac{\vdash \{I \wedge b\} c \{I\}}{\vdash \{I\} \text{ while } b \text{ do } c \{I \wedge \neg b\}} \text{ While}$$

Everything hinges on I . But the rule is not **syntax-directed** in I : it does not compute the invariant, it only **checks** one you supply.

Finding I is the creative act of program verification.

It is the one step no calculus performs for you.

What an invariant must do

To prove $\{P\}$ while b do c $\{Q\}$ with invariant I , three obligations:

1. **Establish:** $P \Rightarrow I$ (I holds on entry)
2. **Preserve:** $\{I \wedge b\} c \{I\}$ (the body maintains it)
3. **Conclude:** $I \wedge \neg b \Rightarrow Q$ (on exit, I gives you Q)

What an invariant must do

To prove $\{P\}$ while b do c $\{Q\}$ with invariant I , three obligations:

1. **Establish:** $P \Rightarrow I$ (I holds on entry)
2. **Preserve:** $\{I \wedge b\} c \{I\}$ (the body maintains it)
3. **Conclude:** $I \wedge \neg b \Rightarrow Q$ (on exit, I gives you Q)

An invariant is a **generalisation of the postcondition** that is also **inductive**. Both halves matter: strong enough to give Q at the end, stable enough to survive a lap.

A worked example: summing 0 to $n - 1$

$$\{n \geq 0\} \quad i := 0; \quad s := 0; \quad \text{while } i < n \text{ do } (s := s + i; \quad i := i + 1) \quad \{ s = \sum_{k=0}^{n-1} k \}$$

A worked example: summing 0 to $n - 1$

$\{n \geq 0\} \quad i := 0; \quad s := 0; \quad \text{while } i < n \text{ do } (s := s + i; \quad i := i + 1) \quad \{ s = \sum_{k=0}^{n-1} k \}$

Try the postcondition itself as the invariant: $s = \sum_{k=0}^{n-1} k$.

A worked example: summing 0 to $n - 1$

$\{n \geq 0\} \quad i := 0; \quad s := 0; \quad \text{while } i < n \text{ do } (s := s + i; \quad i := i + 1) \quad \{ s = \sum_{k=0}^{n-1} k \}$

Try the postcondition itself as the invariant: $s = \sum_{k=0}^{n-1} k$. **Not preserved** — it mentions n , not the running counter.

A worked example: summing 0 to $n - 1$

$$\{n \geq 0\} \quad i := 0; \quad s := 0; \quad \text{while } i < n \text{ do } (s := s + i; \quad i := i + 1) \quad \{s = \sum_{k=0}^{n-1} k\}$$

Try the postcondition itself as the invariant: $s = \sum_{k=0}^{n-1} k$. **Not preserved** — it mentions n , not the running counter. The fix: **generalise n to i** .

$$I \equiv s = \sum_{k=0}^{i-1} k \wedge i \leq n$$

The postcondition is the special case $i = n$. The second conjunct keeps i in range.

Checking the three obligations

$$I \equiv s = \sum_{k=0}^{i-1} k \wedge i \leq n$$

- **Establish.** After $i := 0; s := 0$: $s = 0 = \sum_{k=0}^{-1} k$ (empty sum), and $0 \leq n$ since $n \geq 0$. ✓

Checking the three obligations

$$I \equiv s = \sum_{k=0}^{i-1} k \wedge i \leq n$$

- **Establish.** After $i := 0; s := 0$: $s = 0 = \sum_{k=0}^{-1} k$ (empty sum), and $0 \leq n$ since $n \geq 0$. ✓
- **Preserve.** Assume $I \wedge i < n$. After $s := s + i$: $s = \sum_{k=0}^i k$. After $i := i + 1$: $s = \sum_{k=0}^{i-1} k$ again, and $i \leq n$ since $i < n$ before. ✓

Checking the three obligations

$$I \equiv s = \sum_{k=0}^{i-1} k \wedge i \leq n$$

- **Establish.** After $i := 0; s := 0$: $s = 0 = \sum_{k=0}^{-1} k$ (empty sum), and $0 \leq n$ since $n \geq 0$. ✓
- **Preserve.** Assume $I \wedge i < n$. After $s := s + i$: $s = \sum_{k=0}^i k$. After $i := i + 1$: $s = \sum_{k=0}^{i-1} k$ again, and $i \leq n$ since $i < n$ before. ✓
- **Conclude.** $I \wedge i \geq n$: with $i \leq n$ this forces $i = n$, so $s = \sum_{k=0}^{n-1} k$. ✓

Checking the three obligations

$$I \equiv s = \sum_{k=0}^{i-1} k \wedge i \leq n$$

- **Establish.** After $i := 0; s := 0$: $s = 0 = \sum_{k=0}^{-1} k$ (empty sum), and $0 \leq n$ since $n \geq 0$. ✓
- **Preserve.** Assume $I \wedge i < n$. After $s := s + i$: $s = \sum_{k=0}^i k$. After $i := i + 1$: $s = \sum_{k=0}^{i-1} k$ again, and $i \leq n$ since $i < n$ before. ✓
- **Conclude.** $I \wedge i \geq n$: with $i \leq n$ this forces $i = n$, so $s = \sum_{k=0}^{n-1} k$. ✓

Notice: two of the three obligations are pure **logic**. Only “preserve” touches the program.

Diagnosing a candidate invariant

Given a loop and a candidate I , it is always exactly one of:

1. **Not established.** P doesn't give I . (I too strong at entry.)
2. **Not preserved.** The body can falsify it. (I too strong, or just wrong.)
3. **Preserved but too weak.** Survives every lap, but $I \wedge \neg b$ does not give Q .
4. **Adequate.** Preserved, and strong enough on exit.

Diagnosing a candidate invariant

Given a loop and a candidate I , it is always exactly one of:

1. **Not established.** P doesn't give I . (I too strong at entry.)
2. **Not preserved.** The body can falsify it. (I too strong, or just wrong.)
3. **Preserved but too weak.** Survives every lap, but $I \wedge \neg b$ does not give Q .
4. **Adequate.** Preserved, and strong enough on exit.

The trap

An invariant can be **true of every reachable state** and still useless — if it is too weak to entail Q on exit.

Weakest preconditions

Mechanising the backward calculation

Last time, the swap proof ran backwards, and each assertion was **computed**, not guessed. Let us name the thing being computed.

Weakest precondition

$\text{wp}(c, Q)$ is the **weakest** assertion P such that $\{P\} c \{Q\}$ holds.

Mechanising the backward calculation

Last time, the swap proof ran backwards, and each assertion was **computed**, not guessed. Let us name the thing being computed.

Weakest precondition

$\text{wp}(c, Q)$ is the **weakest** assertion P such that $\{P\} c \{Q\}$ holds.

“Weakest” = most permissive = demands the least. Formally:

$$\models \{P\} c \{Q\} \quad \text{iff} \quad P \Rightarrow \text{wp}(c, Q)$$

Mechanising the backward calculation

Last time, the swap proof ran backwards, and each assertion was **computed**, not guessed. Let us name the thing being computed.

Weakest precondition

$\text{wp}(c, Q)$ is the **weakest** assertion P such that $\{P\} c \{Q\}$ holds.

“Weakest” = most permissive = demands the least. Formally:

$$\models \{P\} c \{Q\} \quad \text{iff} \quad P \Rightarrow \text{wp}(c, Q)$$

So wp turns a **verification** question into an **implication** — the kind of thing you hand to a solver.

The wp calculus (loop-free)

$$\text{wp}(\text{skip}, Q) = Q$$

$$\text{wp}(x := a, Q) = Q[a/x] \quad \leftarrow \text{the assignment axiom, as a function}$$

$$\text{wp}(c_1; c_2, Q) = \text{wp}(c_1, \text{wp}(c_2, Q))$$

$$\text{wp}(\text{if } b \text{ then } c_1 \text{ else } c_2, Q) = (b \Rightarrow \text{wp}(c_1, Q)) \wedge (\neg b \Rightarrow \text{wp}(c_2, Q))$$

The wp calculus (loop-free)

$$\text{wp}(\text{skip}, Q) = Q$$

$$\text{wp}(x := a, Q) = Q[a/x] \quad \leftarrow \text{the assignment axiom, as a function}$$

$$\text{wp}(c_1; c_2, Q) = \text{wp}(c_1, \text{wp}(c_2, Q))$$

$$\text{wp}(\text{if } b \text{ then } c_1 \text{ else } c_2, Q) = (b \Rightarrow \text{wp}(c_1, Q)) \wedge (\neg b \Rightarrow \text{wp}(c_2, Q))$$

Purely **syntax-directed**: recurse on c , right to left, and the precondition falls out. **No invention.**

The six rules of Lecture 8, read as an algorithm.

But loops break the recursion

There is no clause for while.

$\text{wp}(\text{while } b \text{ do } c, Q)$ is a **fixed point** — it unfolds to

$$(\neg b \wedge Q) \vee (b \wedge \text{wp}(c, \text{wp}(\text{while } \dots, Q))) \vee \dots$$

an **unbounded** disjunction over the number of iterations. Not computable by simple recursion.

But loops break the recursion

There is no clause for while.

$\text{wp}(\text{while } b \text{ do } c, Q)$ is a **fixed point** — it unfolds to

$$(\neg b \wedge Q) \vee (b \wedge \text{wp}(c, \text{wp}(\text{while } \dots, Q))) \vee \dots$$

an **unbounded** disjunction over the number of iterations. Not computable by simple recursion.

This is why we annotate

The tool cannot find the loop's precondition by calculation. So **you** supply the invariant, and the tool **checks** it. The human does the creative step; the machine does the bookkeeping.

wp distributes over conjunction:

$$\text{wp}(c, Q_1 \wedge Q_2) = \text{wp}(c, Q_1) \wedge \text{wp}(c, Q_2)$$

Useful: you can verify a conjunctive specification **componentwise**.

Verification conditions

From annotations to formulas

An **annotated program** is a program with an invariant written at each loop.

Verification-condition generation (VCgen) walks the annotated program and emits a finite set of **purely logical** formulas — the VCs — such that:

all VCs valid $\iff$ **the program meets its specification.**

From annotations to formulas

An **annotated program** is a program with an invariant written at each loop.

Verification-condition generation (VCgen) walks the annotated program and emits a finite set of **purely logical** formulas — the VCs — such that:

all VCs valid $\iff$ **the program meets its specification.**

For an annotated loop $\{I\}$ while b do c inside $\{P\} \cdots \{Q\}$, the three VCs are exactly our three obligations:

$$P \Rightarrow I \qquad I \wedge b \Rightarrow \text{wp}(c, I) \qquad I \wedge \neg b \Rightarrow Q$$

The division of labour

VC generation	VC discharge
walk the program, apply wp syntax-directed, mechanical always terminates calculation	prove the resulting implications search / decision procedure the hard part logic
done by the tool	done by an SMT solver

The division of labour

VC generation	VC discharge
walk the program, apply wp syntax-directed, mechanical always terminates calculation	prove the resulting implications search / decision procedure the hard part logic
done by the tool	done by an SMT solver

VCgen produces exactly the $\Rightarrow$ goals; the solver discharges them. Everything hard about automated verification lives in the right-hand column.

VCgen, worked

The annotated program

Back to the summation loop — now **annotated**, and we turn the crank.

$$\{n \geq 0\} \quad i := 0; \quad s := 0; \quad \{I\} \quad \text{while } i < n \text{ do } (s := s + i; \quad i := i + 1) \quad \{s = \sum_{k=0}^{n-1} k\}$$

with the invariant we found earlier:

$$I \equiv s = \sum_{k=0}^{i-1} k \wedge i \leq n$$

The annotated program

Back to the summation loop — now **annotated**, and we turn the crank.

$$\{n \geq 0\} \quad i := 0; \quad s := 0; \quad \{I\} \quad \text{while } i < n \text{ do } (s := s + i; \quad i := i + 1) \quad \{s = \sum_{k=0}^{n-1} k\}$$

with the invariant we found earlier:

$$I \equiv s = \sum_{k=0}^{i-1} k \wedge i \leq n$$

Earlier we **guessed** the three obligations. Now watch them **fall out** of wp.

Step 1: wp of the loop body

The body is $s := s + i$; $i := i + 1$, and we need it to re-establish I .

Compute **right to left**, as always:

$$\text{wp}(i := i + 1, I) = \left(s = \sum_{k=0}^{(i+1)-1} k \wedge i + 1 \leq n \right) = \left(s = \sum_{k=0}^i k \wedge i < n \right)$$

$$\text{wp}(s := s + i, \cdot) = \left(s + i = \sum_{k=0}^i k \wedge i < n \right)$$

Purely **mechanical**: two substitutions. **No invention**.

Step 1: wp of the loop body

The body is $s := s + i$; $i := i + 1$, and we need it to re-establish I .

Compute **right to left**, as always:

$$\text{wp}(i := i + 1, I) = \left(s = \sum_{k=0}^{(i+1)-1} k \wedge i + 1 \leq n \right) = \left(s = \sum_{k=0}^i k \wedge i < n \right)$$

$$\text{wp}(s := s + i, \cdot) = \left(s + i = \sum_{k=0}^i k \wedge i < n \right)$$

Step 1: wp of the loop body

The body is $s := s + i$; $i := i + 1$, and we need it to re-establish I .

Compute **right to left**, as always:

$$\text{wp}(i := i + 1, I) = \left(s = \sum_{k=0}^{(i+1)-1} k \wedge i + 1 \leq n \right) = \left(s = \sum_{k=0}^i k \wedge i < n \right)$$

$$\text{wp}(s := s + i, \cdot) = \left(s + i = \sum_{k=0}^i k \wedge i < n \right)$$

Purely **mechanical**: two substitutions. **No invention**.

Step 2: the three verification conditions

establish

$$n \geq 0 \Rightarrow \left(0 = \sum_{k=0}^{-1} k \wedge 0 \leq n \right)$$

preserve

$$I \wedge i < n \Rightarrow \left(s + i = \sum_{k=0}^i k \wedge i < n \right)$$

exit

$$I \wedge i \geq n \Rightarrow s = \sum_{k=0}^{n-1} k$$

Step 2: the three verification conditions

establish

$$n \geq 0 \Rightarrow \left(0 = \sum_{k=0}^{-1} k \wedge 0 \leq n \right)$$

preserve

$$I \wedge i < n \Rightarrow \left(s + i = \sum_{k=0}^i k \wedge i < n \right)$$

exit

$$I \wedge i \geq n \Rightarrow s = \sum_{k=0}^{n-1} k$$

Three formulas — no program text left. It has been calculated away.

Step 3: discharge them

- **VC 1.** Empty sum is 0, and $n \geq 0$ gives $0 \leq n$. ✓
- **VC 2.** From I we have $s = \sum_{k=0}^{i-1} k$; add i to both sides: $s + i = \sum_{k=0}^i k$. And $i < n$ is assumed. ✓
- **VC 3.** $i \leq n$ (from I) with $i \geq n$ forces $i = n$; substitute. ✓

Step 3: discharge them

- **VC 1.** Empty sum is 0, and $n \geq 0$ gives $0 \leq n$. ✓
- **VC 2.** From I we have $s = \sum_{k=0}^{i-1} k$; add i to both sides: $s + i = \sum_{k=0}^i k$. And $i < n$ is assumed. ✓
- **VC 3.** $i \leq n$ (from I) with $i \geq n$ forces $i = n$; substitute. ✓

Look at what each step needed

Steps 1–2 were **substitution and rearrangement** — a program does that.

Step 3 needed **facts about sums and arithmetic** — that is **logic**.

What a tool does with this

A verifier (Dafny, Verus, Why3, ...) does **exactly** what we just did:

1. read the annotations,
2. run wp backwards to build the VCs,
3. hand each VC to an **SMT solver**.

What a tool does with this

A verifier (Dafny, Verus, Why3, ...) does **exactly** what we just did:

1. read the annotations,
2. run wp backwards to build the VCs,
3. hand each VC to an **SMT solver**.

And when it fails, it tells you *which VC*.

Drop the $i \leq n$ conjunct from I and **VC 3** breaks — you can no longer conclude $i = n$ on exit.

A word on termination

The one thing we are not doing

Recall from Lecture 8: $\{P\}$ while true do skip $\{Q\}$ is valid for *all* P, Q . Partial correctness cannot tell “computes Q ” from “never terminates”.

The one thing we are not doing

Recall from Lecture 8: $\{P\}$ while true do skip $\{Q\}$ is valid for *all* P, Q . Partial correctness cannot tell “computes Q ” from “never terminates”.

Total correctness = partial correctness + termination

Write $[P] \text{ c } [Q]$: if P holds, c **does** terminate *and* Q holds after.

The one thing we are not doing

Recall from Lecture 8: $\{P\}$ while true do skip $\{Q\}$ is valid for *all* P, Q . Partial correctness cannot tell “computes Q ” from “never terminates”.

Total correctness = partial correctness + termination

Write $[P] \text{ c } [Q]$: if P holds, c **does** terminate *and* Q holds after.

The extra ingredient is a **variant**: a quantity that **strictly decreases** each lap and is **bounded below**. A descending sequence of naturals must stop — that is the whole termination argument. (Here: $V = n - i$.)

The one thing we are not doing

Recall from Lecture 8: $\{P\}$ while true do skip $\{Q\}$ is valid for *all* P, Q . Partial correctness cannot tell “computes Q ” from “never terminates”.

Total correctness = partial correctness + termination

Write $[P] \text{ c } [Q]$: if P holds, c **does** terminate *and* Q holds after.

The extra ingredient is a **variant**: a quantity that **strictly decreases** each lap and is **bounded below**. A descending sequence of naturals must stop — that is the whole termination argument. (Here: $V = n - i$.) As a c-ex, while true do skip has **no** variant.

We will not develop the rules — see Winskel §7.

Wrap

- The **invariant** is the one creative step — a generalisation of Q that is also inductive.
- **Weakest preconditions** make the backward calculation an algorithm — for loop-free code. Loops need an annotation, because $\text{wp}(\text{while})$ is a fixed point.
- **VCgen + SMT**: generation is calculation, discharge is logic.
- **Total correctness** adds a well-founded **variant** — mentioned, not developed; it closes the non-termination hole.

Next time

Everything so far assumed variables are **independent boxes**.

Introduce the **heap** (pointers, aliasing) and the assignment axiom **breaks**: after $[x] := a$, you cannot substitute, because you do not know what *else* points at that cell.

Next time

Everything so far assumed variables are **independent boxes**.

Introduce the **heap** (pointers, aliasing) and the assignment axiom **breaks**: after $[x] := a$, you cannot substitute, because you do not know what *else* points at that cell.

Separation logic (Lecture 10): a new connective, $*$, and a new rule — the frame rule — to reason *locally* about a shared, mutable heap.

Questions?